

The Final Capstone Report

A comprehensive guide to convincing reviewers your infrastructure works through executive clarity and technical depth.



Why a Final Report Matters



Design Story

Tells the complete narrative of your design decisions and execution journey



Audit Trail

Provides documented evidence of all key decisions and implementations



Future Reference

Serves as a blueprint for future scaling and system evolution

Your final report is a critical deliverable for both Capstone evaluation and stakeholder sign-off. It demonstrates not just what you built, but how thoughtfully you approached the infrastructure challenge.

Report Structure

A comprehensive 10-section framework that balances business value with technical implementation:

1 Executive Summary

Business value and high-level results

2 Problem Statement & Goals

Clear definition of the challenge and success metrics

3 Architecture Overview

System design and key components

4 Implementation Details

Technical execution specifics

5 Testing & Validation

Evidence of system reliability

1 Monitoring & Cost Management

Operational visibility and efficiency

2 Security & Compliance

Risk mitigation and protection measures

3 Lessons Learned & Risks

Transparency about challenges

4 Future Roadmap

Evolution and enhancement plans

5 Appendices

Supporting documentation and evidence



Executive Summary Template

The Perfect Executive Summary:

- 1-2 pages in **non-technical language**
- Clearly articulates the problem solved and its business significance
- Highlights measurable results: uptime percentages, latency improvements, SLO achievements
- Quantifies business impact with specific metrics
- States definitive production-readiness conclusion

✔ Example Impact Statement

"Our AI infrastructure cut inference latency by 40%, resulting in \$10,000 monthly operational savings while improving customer experience."

Keep executive summary focused on outcomes, not technical details

Problem Statement & Goals

Define the Challenge

- Specify your **AI domain focus** (NLP, computer vision, RAG system, etc.)
- Articulate specific **pain points** of non-scalable infrastructure
- Establish clear **scope boundaries**: training, serving, monitoring

This section sets expectations and provides the yardstick for measuring your success.

Performance Goals

p95 latency < 300ms

Reliability Goals

Availability $\geq$ 99.9%

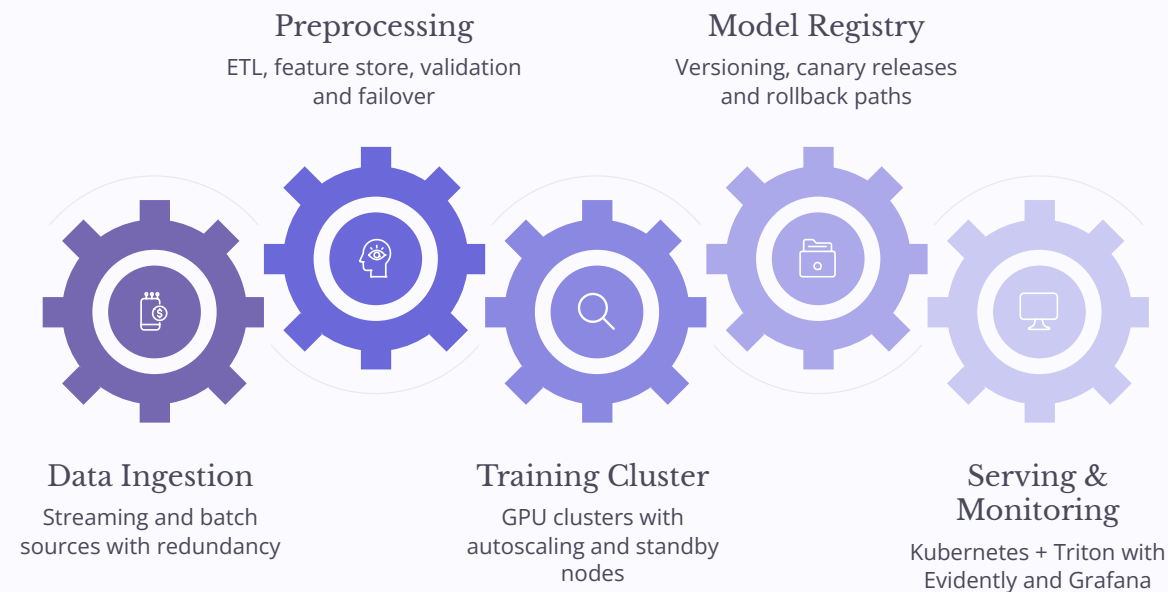
Model Quality Goals

Drift monitoring implemented with alerts

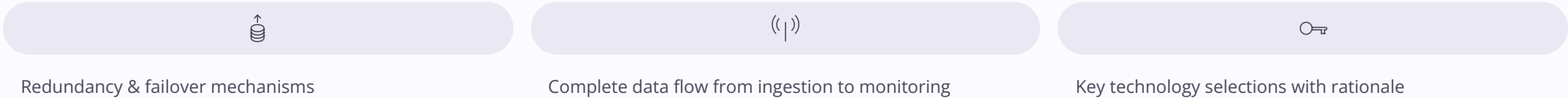
Efficiency Goals

Cost per 1k requests within target budget

Architecture Overview



Your architecture diagram should be the **visual centerpiece** of your report, clearly showing:



Implementation Details



Containerization & Orchestration

Docker images and Kubernetes configuration with resource specifications



CI/CD Pipeline

GitHub Actions workflows, Helm charts, progressive rollout policies



HA & Autoscaling

Horizontal Pod Autoscalers and PodDisruptionBudgets for resilience



Cost Infrastructure

Resource tagging, Kubecost integration, budget alert thresholds



Security Implementation

IAM roles, RBAC policies, mTLS configuration, secrets management



Code References

Repository structure and Architecture Decision Records (ADRs)

This section bridges the gap between architectural vision and technical reality. Include specific configuration details that demonstrate your implementation depth.

Testing & Validation

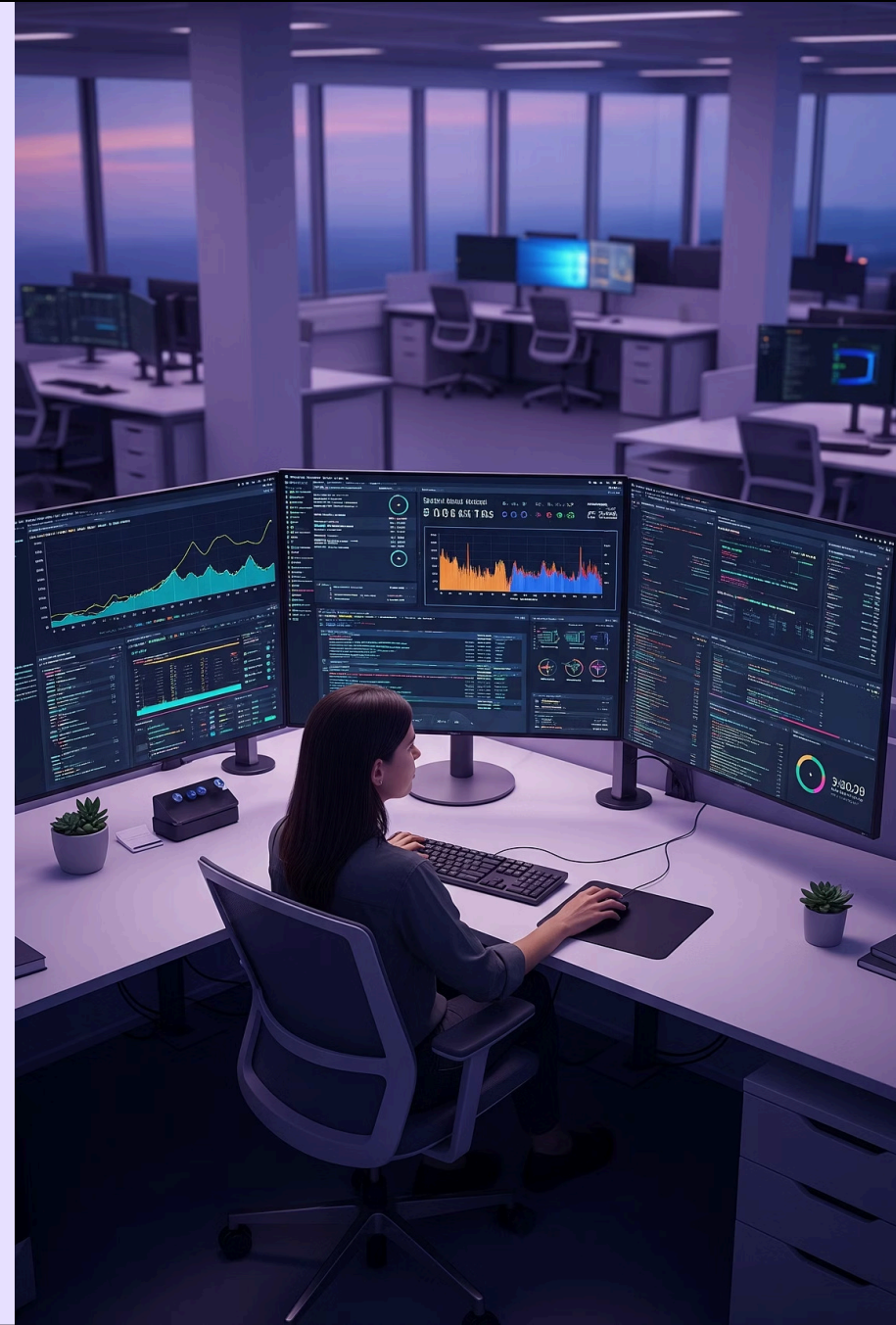
Comprehensive Test Strategy

- End-to-end functional testing coverage
- Load testing with k6/Locust showing throughput limits
- Chaos engineering experiments with Chaos Mesh
- Extended soak tests demonstrating stability

Key Validation Metrics

- System knee point: maximum throughput before degradation
- Failover recovery time and error budget consumption
- Drift detection accuracy using PSI/KS statistics
- Model retraining trigger effectiveness

Pro tip: Include actual screenshots of your test dashboards as concrete evidence of your validation approach.



Monitoring & Cost Management

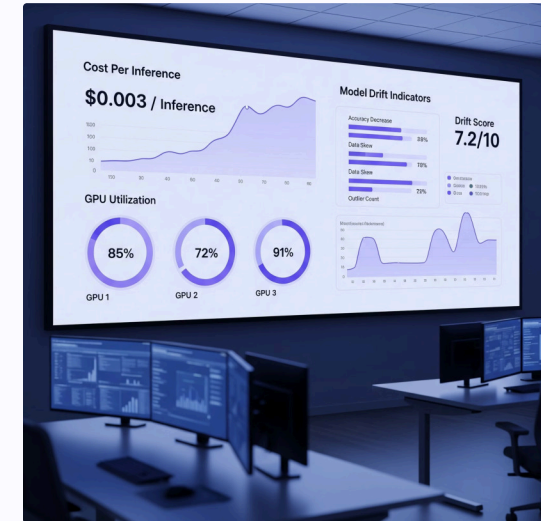
Operational Visibility

Your monitoring infrastructure demonstrates system accountability:

- Custom Grafana dashboards for API health, model drift, and resource utilization
- Alert policies with clear ownership and runbook links
- Comprehensive cost reporting with per-request metrics

Optimization strategies implemented:

- Spot GPU instances for non-critical workloads
- Mixed precision inference to maximize throughput
- Tiered storage strategy for model artifacts



Monitoring Outcome

Your infrastructure is **visible**, **accountable**, and **efficient**, with every dollar spent justified by business value.

Security & Compliance

Identity & Access Management

Comprehensive IAM matrix mapping roles to specific actions with enforcement mechanisms

- Principle of least privilege implementation
- Service account segregation by function

Kubernetes Security

RBAC policies and namespace isolation with pod security standards

- Network policies restricting pod-to-pod communication
- Admission controllers enforcing security policies

Data Protection

Immutable audit logs with distributed tracing for request accountability

- PII handling protocols with appropriate masking
- Encryption for data at rest and in transit

AI Safety Measures

Content filtering systems with explicit refusal rate logging for LLMs

- Prompt injection detection mechanisms
- Regular red team exercises to identify vulnerabilities

Lessons Learned

Technical Challenges

Scaling Bottlenecks

Network throughput limitations required architecture adjustments to prevent GPU idle time

Monitoring Gaps

Initial blind spots in observability led to adding comprehensive drift metrics

HA Cost Tradeoffs

Balancing high availability requirements against budget constraints required creative solutions

Being transparent about challenges demonstrates maturity and provides valuable insights for future implementations.

Process Improvements



Team workflows that enhanced delivery:

- Documentation-as-code approach
- CI/CD with ephemeral environments
- Regular architecture reviews

Risk Register

Identifying and addressing potential vulnerabilities is critical for production readiness.

Current Risks

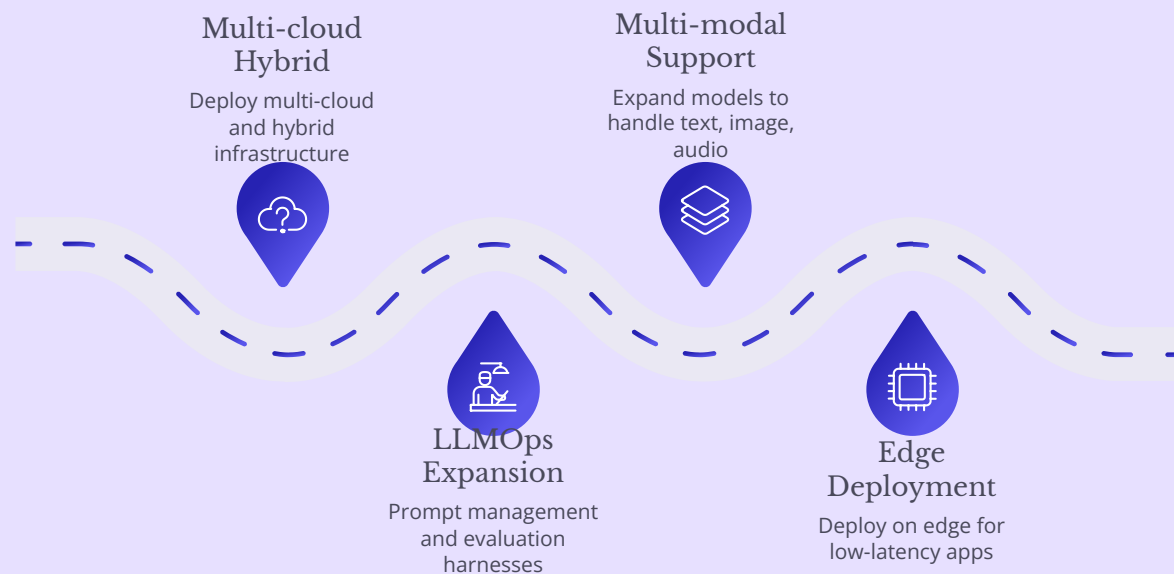
- Model extraction via carefully crafted queries
- Cross-region failover costs exceeding budget constraints
- Extended retraining cycles during severe drift scenarios
- Knowledge gaps in specialized infrastructure components
- Potential vendor lock-in with cloud-specific services

Mitigation Strategies

- Implementing query rate limits and watermarking techniques
- Evaluating active-passive configuration as cost-effective alternative
- Developing smaller checkpoint retraining strategy for incremental updates
- Cross-training team members on critical infrastructure components
- Abstracting vendor-specific APIs behind internal interfaces

Each identified risk should include severity rating, likelihood, and owner responsible for mitigation.

Future Roadmap



Near-term opportunities (6-12 months):

- **Multi-cloud/hybrid infrastructure** to reduce vendor dependency
- **LLMOps feature expansion** with prompt management and evaluation harnesses

Long-term vision (12-24 months):

- **Multi-modal model support** for text, image, and audio processing
- **Edge device deployment** for latency-critical applications
- **Green AI initiatives** to optimize energy usage and carbon footprint

Appendices

Comprehensive supporting documentation provides depth and credibility to your main report.



Architecture Decision Records

Documented rationale for key technical choices, alternatives considered, and constraints that guided decisions



API Documentation

Complete OpenAPI specification detailing all endpoints, parameters, responses, and error handling



Model & Data Cards

Detailed documentation of model characteristics, training datasets, and performance characteristics



Operational Artifacts

Drift reports, Grafana dashboard exports, stress test results, and CI pipeline logs



Writing Style Guidelines

1 Use Active Voice

Write in concise, direct sentences that clearly attribute actions

- ✓ "The system processes 1,000 requests per second"
- "1,000 requests per second are processed by the system"

2 Visualize Data

Use tables, charts, and diagrams instead of text-heavy descriptions

Break up dense technical information with visual elements to improve readability

3 Validate Every Metric

Include source and time window for all performance claims

"p99 latency of 150ms measured across 10M requests during 7-day soak test"

Audience-Appropriate Content

Executive Summary

Written for managers and stakeholders who need business outcomes without technical details

Technical Sections

Written for engineers and SREs who need implementation specifics and technical rationale

Document Ownership

Include clear ownership and update information:

- Section owner's name and contact
- Last update timestamp
- Review cycle information

Tools to Generate Report



MkDocs / Docusaurus

Static site generators that convert Markdown into professional documentation websites



Jupyter + nbconvert

Interactive notebooks for including live test results and data visualizations



CI Pipeline

Automated build process to publish updated documentation on every code change



Pandoc

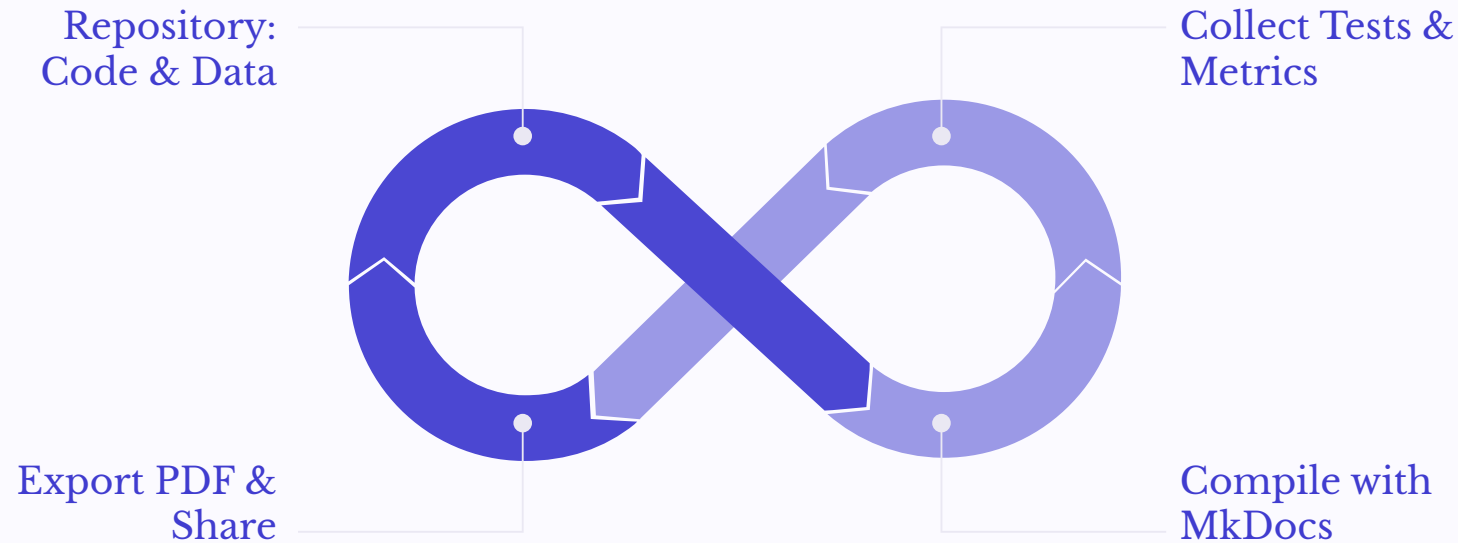
Universal document converter for exporting to PDF, DOCX, and other formats



Mermaid/PlantUML

Code-based diagramming tools that generate architecture visuals from text descriptions

Report Assembly Workflow



Docs-as-code ensures your report is always current with the latest implementation details.

This approach treats documentation as a first-class citizen in your development process, ensuring that report contents stay synchronized with actual infrastructure changes. By keeping documentation in the same repository as code, you create a natural update cycle that maintains accuracy.

Capstone Checklist

Use this comprehensive checklist to ensure your report meets all requirements before submission:

- **Repository Structure**

Report folder organized with clear navigation and structure

- **Core Content**

Executive summary and problem/goals sections written and reviewed

- **Architecture Documentation**

System diagram completed with supporting ADRs included

- **Evidence Collection**

Testing results and monitoring screenshots attached

- **Risk Management**

Security measures and compliance requirements documented

- **Financial Aspects**

Cost analysis and optimization strategies outlined

- **Forward Planning**

Future roadmap defined with clear next steps

- **Final Deliverable**

Report exported to PDF/HTML format for submission

Each item should be independently verified before final submission to ensure completeness and quality.



Summary: The Power of Your Report

Narrative + Evidence

Your report combines compelling storytelling with concrete technical evidence to demonstrate your infrastructure's effectiveness

Comprehensive Coverage

Architecture, testing, monitoring, security, and cost analysis work together to present a complete picture of your solution

Documentation as Code

Versioned, reviewable, and reproducible documentation ensures accuracy and builds reviewer confidence

A well-crafted final report positions you for **successful Capstone defense** and demonstrates your solution's **production readiness**.

Discussion Question

"Would your Capstone report focus more on technical resilience or business ROI? Why?"

Technical Resilience Focus

Emphasizes system reliability through:

- High availability architecture
- Drift detection mechanisms
- Comprehensive stress testing
- Failure recovery protocols

Appeals to technical evaluators concerned with system stability and operational excellence

Business ROI Focus

Highlights value creation through:

- Cost savings calculations
- Performance improvements
- Uptime SLA guarantees
- Business impact metrics

Appeals to stakeholders concerned with financial justification and business outcomes

The ideal report balances both perspectives, connecting technical excellence to tangible business value.